AIM 1: To understand tokens in C++ such as keywords, identifiers, and constants.

Aim 2: To learn different types of constants such as integer, real, character, string, and backslash constants.

Aim 3: To study the features of C++ and its basic structure.

Aim 4: To write and execute a simple C++ program without using classes.

PROGRAM :

```cpp
#include <iostream>
using namespace std;

int main(){
    string name = "Bilal Patel";
    int rollNum = 16;
    char grade = 'O';
    float marks = 90.9;
    double pie = 3.14;
    int age = 18;

    cout<<"STUDENT NAME : "<< name
        <<endl;
    cout<<"STUDENT ROLL No. :"<<rollNum
        <<endl;
    cout<<"STUDENT GRADE : "<< grade
        <<endl;
    cout<<"STUDENT MARKS : "<< marks
        <<endl;
    cout<<"STUDENT AGE  : "<< age<<endl
        ;
    cout<<"Value of Pie  : "<< pie
        <<endl;
    return 0;
}
```

OUTPUT :

```
STUDENT NAME : Bilal Patel
STUDENT ROLL No. :16
STUDENT GRADE : O
STUDENT MARKS : 90.9
STUDENT AGE  : 18
Value of Pie  : 3.14


=== Code Execution Successful ===
```

AIM 6: Write a program to demonstrate implicit and explicit type casting.

PROGRAM :

```cpp
#include <iostream>
using namespace std;

int main() {

    int a = 10;
    float b = a;

    cout << "Implicit Casting (int ? float)" << endl;
    cout << "Value of a(int): " << a << endl;
    cout << "Value of b(float): " << b << endl;

    float x = 9.87f;
    int y = int(x);

    cout << "\nExplicit Casting (float ? int):" << endl;
    cout << "Value of x(float): " << x << endl;
    cout << "Value of y(int): " << y << endl;

    return 0;
}
```

OUTPUT :

```
Implicit Casting (int ? float)
Value of a(int): 10
Value of b(float): 10

Explicit Casting (float ? int):
Value of x(float): 9.87
Value of y(int): 9


=== Code Execution Successful ===
```

AIM 7: Write a program to use cin and cout for input and output operations.

PROGRAM :

```cpp
#include <iostream>
using namespace std;

int main() {
    string name;
    float marks;
    int age;

    cout << "ENTER STUDENT NAME: ";
    cin >> name;

    cout << "ENTER STUDENT MARKS: ";
    cin >> marks;

    cout << "ENTER STUDENT AGE: ";
    cin >> age;

    cout << "\n----output----\n";
    cout << "STUDENT NAME : " << name << endl;
    cout << "STUDENT MARKS : " << marks << endl;
    cout << "STUDENT AGE : " << age << endl;

    return 0;
}
```

OUTPUT :

```
ENTER STUDENT NAME: Bilal
ENTER STUDENT MARKS: 85
ENTER STUDENT AGE: 18

----output----
STUDENT NAME : Bilal
STUDENT MARKS : 85
STUDENT AGE : 18


=== Code Execution Successful ===
```

Aim 8 : Write a program to demonstrate different operations in C++.

PROGRAM :

```cpp
#include <iostream>
using namespace std;

int main() {

    int a = 10, b = 5, c = 2;

    int result1 = a + b * c;
    int result2 = (a + b) * c;
    int result3 = a / b * c;
    int result4 = a + b - c * 3;

    cout << "---Operator Precedence Demonstration---" << endl;
    cout << "a + b * c = " << result1 << endl;
    cout << "(a + b) * c = " << result2 << endl;
    cout << "a / b * c = " << result3 << endl;
    cout << "a + b - c * 3 = " << result4 << endl;

    return 0;
}
```

OUTPUT :

```
---Operator Precedence Demonstration---
a + b * c = 20
(a + b) * c = 30
a / b * c = 4
a + b - c * 3 = 9


=== Code Execution Successful ===
```

Aim 9 :To write a program to demonstrate different function types in C++.

PROGRAM :

```cpp
#include <iostream>
using namespace std;

void show1() {
    cout << "Internal Function\n";
}

inline void show2() {
    cout << "Inline Function\n";
}

extern void show3() {
    cout << "External Function";
}

int main() {
    show1();
    show2();
    show3();
    return 0;
}
```

OUTPUT :

```
Internal Function
Inline Function
External Function

=== Code Execution Successful ==
```

AIM 10 : To write a program in C++ to demonstrate the class and object.

PROGRAM :

```cpp
#include <iostream>
using namespace std;

class Student {
public:
    int a = 16;
    void show() {
        cout << " Bilal Patel";
    }
};

int main() {
    Student s;
    cout << s.a;
    s.show();
    return 0;
}
```

OUTPUT :

```
16 Bilal Patel

=== Code Execution Successf
```

AIM 11 : To write a program in C++ to demonstrate the concept of pass by value and pass by reference.

PROGRAM :

```cpp
#include <iostream>
using namespace std;

class Student {
public:
    void show(int a) {
        a = a + 10;
        cout << a << endl;
    }

    void show1(int &a) {
        a = a + 10;
        cout << a << endl;
    }
};

int main() {

    Student s;

    int x = 5;
    s.show(x);

    int y = 10;
    s.show1(y);

    return 0;
}
```

OUTPUT :

```
15
20


=== Code Execution Successful ===
```

AIM 12 : To study and implement different types of constructor and destructor in C++.

PROGRAM :

```cpp
#include <iostream>
using namespace std;

class A {
public:
    A() {
        cout << "Default Constructor" << endl;
    }

    A(int x) {
        cout << "Parameter Constructor" << endl;
    }

    A(A &obj) {
        cout << "Copy Constructor" << endl;
    }

    ~A() {
        cout << "Destructor" << endl;
    }
};

int main() {
    A a;
    A b(2);
    A c(b);
    return 0;
}
```

OUTPUT:

```
Default Constructor
Parameter Constructor
Copy Constructor
Destructor
Destructor
Destructor


=== Code Execution Successful ==
```

AIM 13 : To study and implement various concepts of working with object including accessor and mutator function, static data member, static function, access modifiers and array of object.

PROGRAM :

```cpp
#include <iostream>
#include <string>
using namespace std;

class A {
private:
    string name;
    static int roll;

public:
    void setName(string x) {
        name = x;
    }

    string getName() {
        return name;
    }

    static int getRoll() {
        return roll;
    }
};

int A::roll = 101;

int main() {
    A a[2];
    a[0].setName("Bilal Patel\n ");
    a[1].setName("Faiz Khan ");

    cout << a[0].getName();
    cout << a[1].getName();

    return 0;
}
```

OUTPUT :

```
Bilal Patel
 Faiz Khan

=== Code Execution Successful ==
```

AIM 14 : To study and implement the concept of polymorphism

(a) Function Overloading.

PROGRAM :

```cpp
#include <iostream>
using namespace std;

class A {
public:
    void add(int x, int y) {
        int z = x + y;
        cout << z << endl;
    }

    void add(double x, double y) {
        double z = x + y;
        cout << z << endl;
    }
};

int main() {
    A a;
    a.add(1, 4);
    a.add(1.2, 3.79);
    return 0;
}
```

OUTPUT :

```
5
4.99


=== Code Execution Successful =
```

(B) unary operator overloading :

PROGRAM :

```cpp
#include <iostream>
using namespace std;

class Demo {
public:
    int a;

    void operator++() {
        a = a + 1;
    }

    void show() {
        cout << a;
    }
};

int main() {
    Demo d;
    d.a = 5;
    ++d;
    d.show();
    return 0;
}
```

OUTPUT :

```
6

=== Code Execution Succes
```

(C) Binary operator overloading:

PROGRAM:

```cpp
#include <iostream>
using namespace std;

class Demo {
public:
    int a;

    void getData() {
        cout << "Enter a number: ";
        cin >> a;
    }

    Demo operator+(Demo bb) {
        Demo cc;
        cc.a = a + bb.a;
        return cc;
    }

    void putData() {
        cout << "\nvalue= " << a;
    }
};

int main() {
    Demo aa, bb, cc;
    aa.getData();
    bb.getData();
    cc = aa + bb;

    aa.putData();
    bb.putData();
    cc.putData();

    return 0;
}
```

OUTPUT:

```
Enter a number: 2
Enter a number: 4

value= 2
value= 4
value= 6

=== Code Execution Successf
```

AIM 15 :To study and implement the concept of inheritance.

(a) Single inheritance

PROGRAM :

```cpp
#include <iostream>
using namespace std;

class Person {
public:
    void showPerson() {
        cout << "show person\n";
    }
};

class Student : public Person {
public:
    void showStudent() {
        cout << "show student";
    }
};

int main() {
    Person p;

    Student s;
    s.showPerson();
    s.showStudent();
    return 0;
}
```

OUTPUT :

```
show person
show student

=== Code Execution Successful ===
```

(B) Multilevel inheritance :

PROGRAM :

```cpp
#include <iostream>
using namespace std;

class Person {
public:
    void showPerson() {
        cout << "show person" << endl;
    }
};

class Student : public Person {
public:
    void showStudent() {
        cout << "show student" << endl;
    }
};

class Library : public Student {
public:
    void showLibrary() {
        cout << "show library" << endl;
    }
};

int main() {
    Library l;

    l.showPerson();
    l.showStudent();
    l.showLibrary();

    return 0;
}
```

OUTPUT :

```
show person
show student
show library



=== Code Execution Successful
```